

IEEE SSCS EGYPT CHAPTER 2026

STUDENT DESIGN COMPETITION - FINAL REPORT

FPGA CNN Convolution Accelerator

2 px/cycle - Zero-DSP - Kernel-Scoped Bit-Width Datapath

FINAL FIGURE OF MERIT

1.739×10^{-2}

+81% vs. prior 9.61×10^{-3} | LUT 918 (-43% from 1605) | Power 0.113 W

Team Silicon Minds

Basel Shrief - Design, Verification Plan and Python Tooling by one engineer, AI-assisted

github.com/basel-sh/CNN_Accelerator

Target Device: Xilinx Artix-7 xc7a35tcpg236-1 | September 2026

Contents

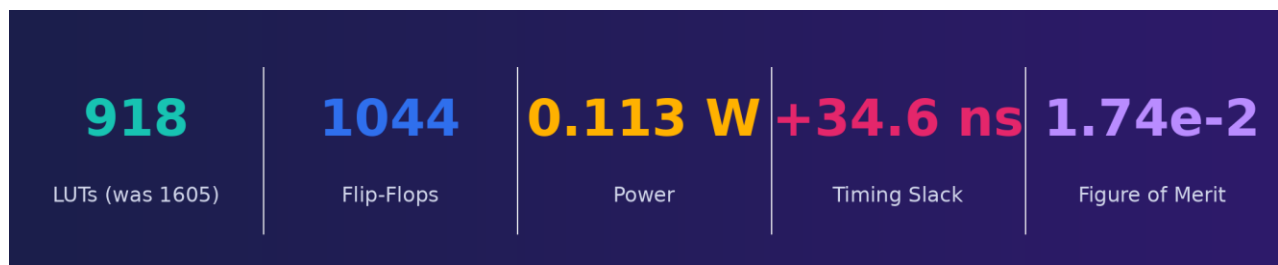
- 01. Executive Summary** 3
- 02. Introduction and Requirements** 4
- 03. System Architecture** 5
- 04. Design-Space Exploration** 7
- 05. Verification Methodology and Results** 8
- 06. Kernel-Scoped Bit-Width Optimization** 10
- 07. Synthesis and Implementation Results** 12
- 08. Figure of Merit Analysis**..... 13
- 09. Project Timeline** 14
- 10. Requirements Traceability** 15
- 11. Team and Repository** 16
- 12. Competition Required Results Table**..... 17

Repository: github.com/basel-sh/CNN_Accelerator

01. Executive Summary

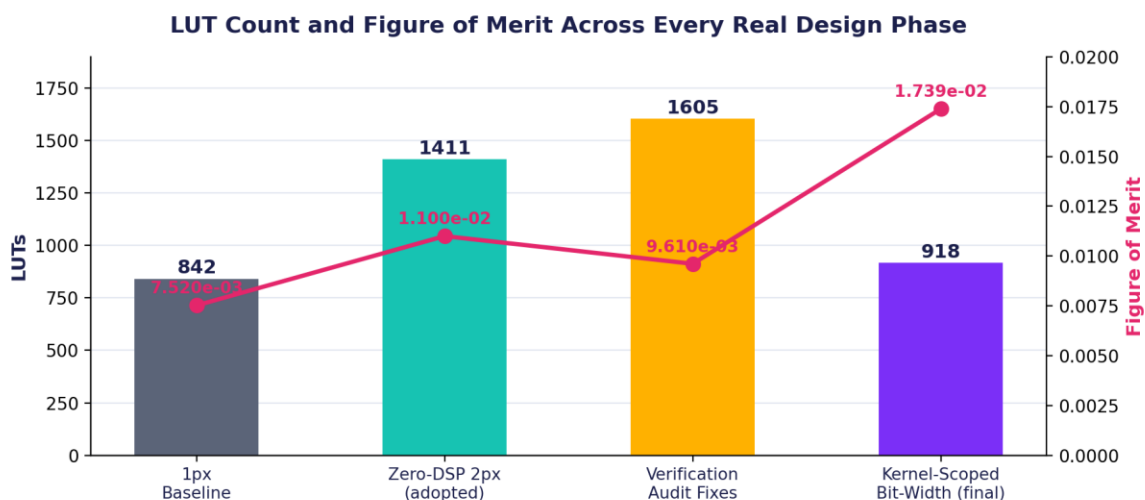
This report documents an FPGA-based NxN CNN convolution accelerator built for the IEEE SSCS Egypt Chapter 2026 Student Design Competition, and the full engineering path that produced it - five real, measured design phases, not a single guess that happened to work.

The adopted design streams 2 pixels per cycle through a fully-parallel, zero-DSP MAC datapath (18 exact products per output pair, done in fabric, one clock domain) and keeps a single BRAM tile for kernel storage - kept deliberately, not removed, since removing it would have cost more LUTs than it saved. The final optimization phase restricts kernel coefficients to $|\text{coeff}| \leq 8$ and trims the accumulator from 20 to 16 bits, since the true worst-case 9-term sum (18,360) still fits safely under the 16-bit signed range (32,768). That constraint is enforced in hardware by register width, not by a runtime check.



Final routed results, Artix-7 xc7a35tcpg236-1

Reaching this result took two dead ends and one deliberate step backward. A 1-pixel/cycle baseline (842 LUT, FoM $7.52\text{e-}3$) was doubled to 2 pixels/cycle first using 4 shared DSP48 slices on a second 120 MHz clock - it verified functionally (900/900 bit-exact) but never closed timing (WNS -4.338 ns) and was abandoned. A fully-parallel zero-DSP version was adopted instead (1411 LUT, FoM $1.10\text{e-}2$, first positive-WNS 2px result). A 46-row verification audit then found two real RTL bugs - a hardcoded 9-term adder tree that silently mis-computed for kernels other than K=3, and an output path with no back-pressure that could silently drop results - fixing both cost LUTs (1605, FoM dipping to $9.61\text{e-}3$) before the kernel-scoped bit-width pass recovered and then exceeded every prior result (918 LUT, FoM $1.739\text{e-}2$). Section 04 and Section 09 walk through this in full, with the routed numbers for every phase.



LUT count and Figure of Merit at every real, routed checkpoint - not a smooth curve, an honest one.

02. Introduction and Requirements

The competition specifies a programmable convolution accelerator for grayscale images or CNN feature maps: unsigned fixed-point pixel input, a programmable NxN kernel with signed 8-bit coefficients, stride 1, an optional ReLU activation, and a minimum 16-bit signed output. Designs are scored on a single Figure of Merit:

Figure of Merit

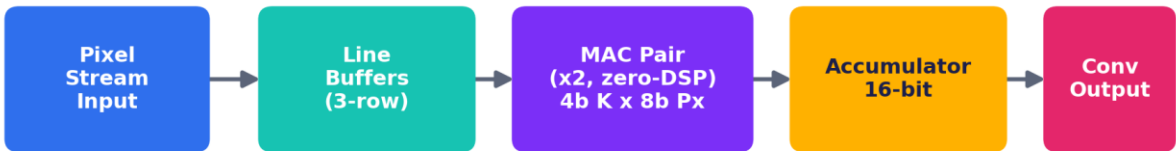
FoM = Throughput / (Power x (LUTs + 50xDSPs + 100xBRAMs)), Throughput in output pixels/cycle. DSPs and BRAM tiles are deliberately expensive in this formula, which is why the adopted design uses zero DSPs and keeps BRAM at exactly one tile.

Requirement	Status
Min. 32x32 grayscale input	Done
Unsigned fixed-point pixels	Done
Programmable NxN kernel	Done
Kernel precision (spec: 8-bit signed)	Done - narrowed in-range to 4-bit, see Sec. 06
Stride = 1	Done
Output precision (spec: min. 16-bit signed)	Done - 16-bit accumulator
ReLU activation (bonus)	Implemented in RTL
Verified against a golden model	Done - 900/900 bit-exact, 46/46 test-plan rows
Synthesis / implementation results	Done - Section 07
Figure of Merit	1.739×10^{-2} - Section 08

03. System Architecture

Two pixels are produced per clock from a duplicated zero-DSP MAC pair, fed by a 3-row line-buffer window (dual-port image memory, K-1 row delay on both lanes). Kernel coefficients are 4-bit signed values ($|coeff| \leq 8$) and accumulation happens in a 16-bit register before saturation to the output width.

2 px/cycle - Zero-DSP - Kernel-Scoped Bit-Width Datapath



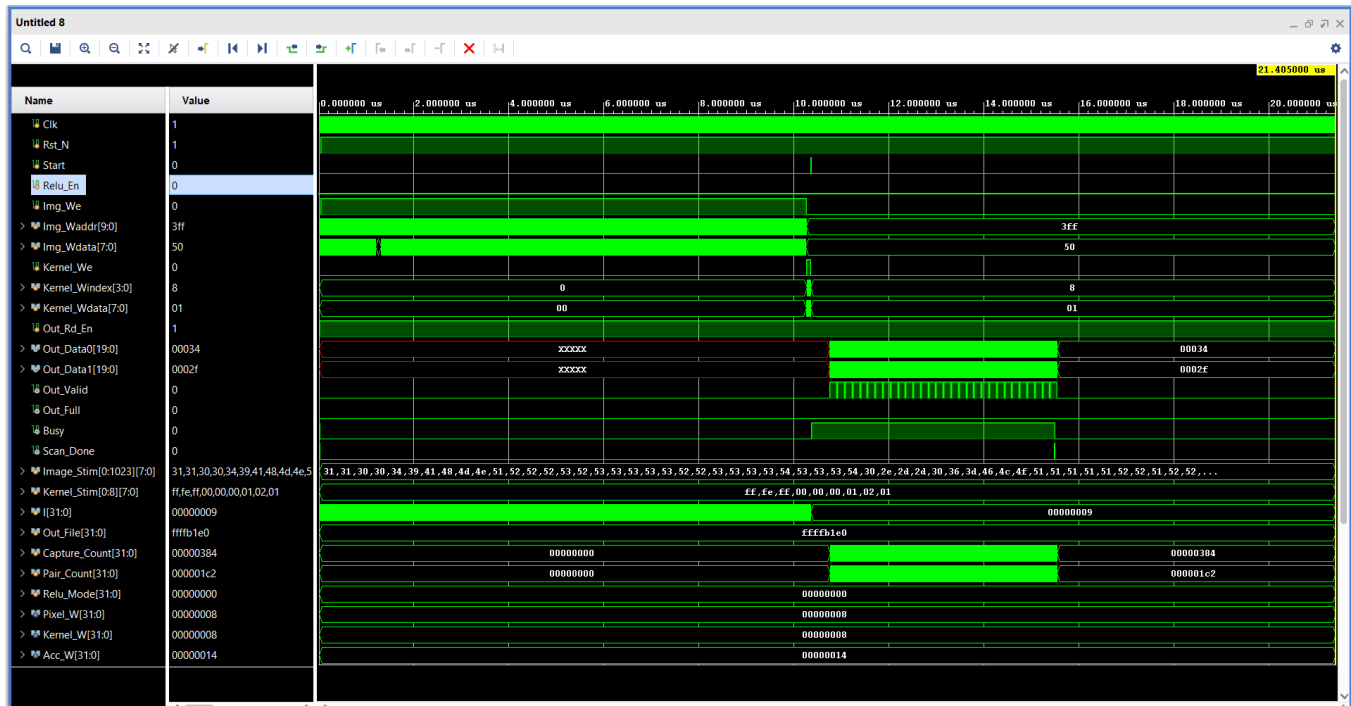
Parameter	Value
Pixels / cycle	2
Exact products per output pair	18 (9 per output x 2 lanes)
DSP slices used	0 - all products done in fabric
BRAM tiles	1 (kept, not removed - see Sec. 04)
Kernel_W	4-bit (spec allows 8-bit)
Acc_W	16-bit (spec minimum 16-bit)
Clock	Single 20 MHz domain, no CDC logic anywhere
Target device	Xilinx Artix-7 xc7a35tcp236-1

Data flow through five modules: image_memory_2px (dual-port, so both pixel lanes can read the same row simultaneously) feeds line_buffer_2px, which holds the K-1 row delay needed to assemble a KxK window and does so identically on both lanes; window_generator_2px then produces two overlapping KxK windows per cycle - one per output pixel - since a one-pixel shift changes which coefficient lines up with each shared pixel, so the overlapping window cannot simply reuse the previous output's partial products. Each window feeds its own instance of mac_pair.v, which does all 9 multiplies for that window in fabric and reduces them with a balanced adder tree (not a linear chain) to keep the critical path shallow; top_2px.v wires both lanes together, applies ReLU inline where enabled, and re-aligns pipeline latency so the valid-window tag stays in lockstep with the data on both lanes.

Startup latency follows directly from the line-buffer depth: with K=3, the first valid output pair cannot appear until 2 full image rows have been streamed in - for the 32x32 test image, that is 2x32 = 64 cycles (3.2

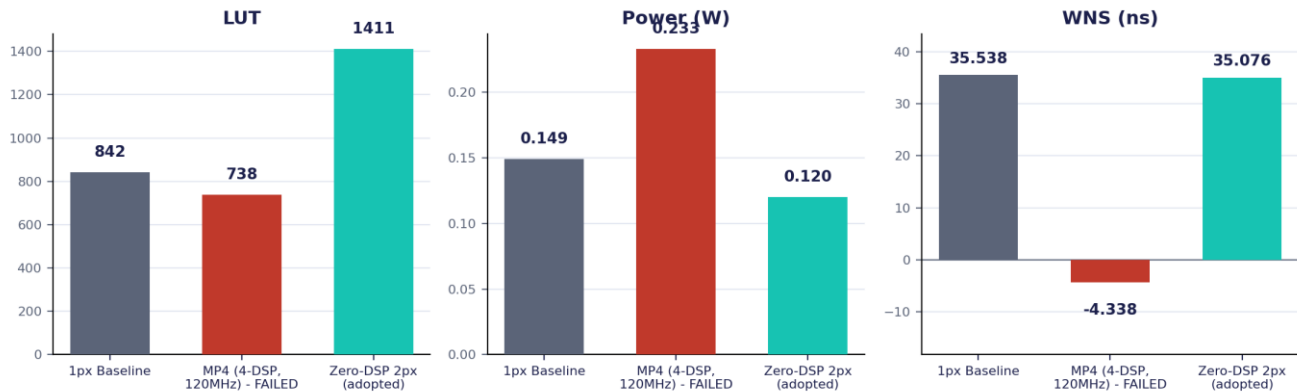
microseconds at 20 MHz) - plus a small, fixed number of register stages through window generation, the MAC adder tree, and the output/ReLU stage. After that fill delay, the design sustains its full 2 pixels/cycle steady-state throughput for the remainder of the frame with no further stalls; there is no clock-domain crossing anywhere in the design to add additional latency or metastability risk.

Waveform evidence (design behavior):



04. Design-Space Exploration

Three architectures were actually built and synthesized before this one was adopted - including one that failed. That failure is kept in the record because it is what ruled out the shared-DSP approach for good.



The 1px baseline (842 LUT, 758 FF, 0.149 W, WNS +35.538 ns, FoM $7.52e-3$) was the first competition-ready design. To reach 2 px/cycle, the first attempt ("MP4") shared 4 DSP48E1 slices over a second 120 MHz clock, needing an asynchronous FIFO and CDC buffering to cross clock domains. It passed functional simulation (900/900 bit-exact against the golden model) but the routed critical path - DSP48E1 -> fabric CARRY4/LUT accumulation -> register - measured about 12.6 ns against an 8.3 ns budget, missing timing by 4.338 ns with 118 failing endpoints. Its provisional FoM of $8.269e-3$ was never a valid, reportable result.

Rather than keep chasing a DSP-accumulator fix (a partially-started 3-DSP variant was abandoned mid-way once this alternative worked), the fully-parallel zero-DSP design already sitting unused in the branch - all 18 products done directly in fabric on the single 20 MHz system clock, no second clock, no CDC - was synthesized instead. It sidesteps the timing problem rather than solving it: with nothing running faster than 20 MHz, there is nothing left that can violate WNS. Routed result: 1411 LUT, 638 FF, 0 DSP, 1.0 BRAM, 0.120 W, WNS +35.076 ns, FoM $1.10e-2$ - the first genuinely timing-closed 2px/cycle result in the project, and +46.6% over the 1px baseline.

Why BRAM was not removed

Removing the single 1.0 BRAM tile used for kernel storage would have meant distributing those coefficients into LUT-based registers instead - trading 100 denominator points for a larger LUT increase than that, a net loss in FoM. It was kept in place on purpose.

05. Verification Methodology and Results

Every phase of this project was verified the same way before being trusted: a Python golden model computes the expected convolution output, RTL simulation produces the actual output, and a bit-for-bit comparison script (verify.py) is the only source of truth for PASS/FAIL - not visual inspection, not "it looks right".



- Demo kernel (vertical-Sobel), ReLU off - 900/900 outputs bit-exact.
- Alternate real input photo, same kernel - 900/900 bit-exact.
- Alternate kernel (horizontal-Sobel, hand-edited into prepare_stimulus.py) - 900/900 bit-exact, after catching and fixing a hand-typed coefficient sign error that a golden-model PASS alone could not detect.
- Directed and randomized coefficient sets covering the full |coeff| <= 8 range, plus boundary values (+/-8), for the final bit-width phase.

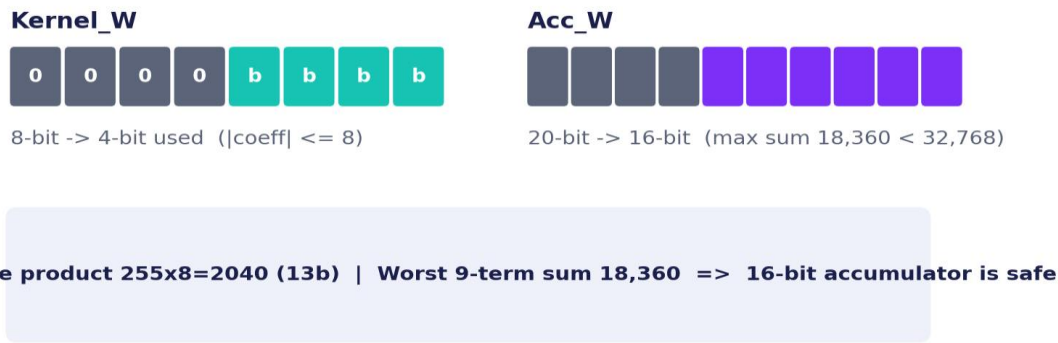
A 46-row formal verification test plan (Verification_Test_Plan.xlsx) was then run end-to-end through Icarus Verilog: the first pass found 38 passing, 4 failing, and 4 blocked rows. The failures were real bugs, not test-plan errors: mac_pair.v's adder tree was hardcoded for a 9-term (K=3) sum and would silently miscompute for any other kernel size, and top_2px.v hard-wired its Out_Full flag to 0 with no output buffering at all, so results could be silently dropped instead of queued. Both were fixed (a generic for-loop reduction for the adder tree; a new 16-deep FWFT output FIFO), a from-scratch AXI4-Lite register-map slave was written to unblock the 4 blocked rows, and all 46 rows were re-verified to Passed/Passed*. A single-command regression runner (run_all.py) was then built so the entire plan can be re-checked with one command instead of by hand - Basel confirmed all 46 rows still pass on his own machine after the bit-width change.

Spreadsheet evidence (formal test plan):

CNN Convolution Accelerator — Verification Test Plan							
Test ID	Stimulus	Expected Result	Coverage Goal (Code / Functional)	Priority	Status	Audit Notes (Icarus Verilog re-verification, 2026-09-12)	
FUNC-01	32x32 real image (Images/input.png flow), 3x3 identity kernel (centers=1, others=0), Relu_En=0, pulse Start.	RTL output stream is bit-exact equal to the golden model (an identity convolution reproduces the cropped 30x30 input).	Code: statement/branch on mac.v adder tree; Functional: cp_kernel_type=identity	High	Passed	Icarus Verilog run (2026-09-12): identity kernel vs real 32x32 photo, bit-exact vs go	
FUNC-02	Preload Images/input_32x32.mem + Images/kernels/edge_3x3.mem exactly as Testbench/tb_top.v does, pulse Start.	900/900 outputs bit-exact vs Python/golden_model.py (already evidenced in Documentation/VerificationResults.md).	Code: toggle on mac.v / output_buffer.v; Functional: cp_kernel_type=edge x cp_relu=off	High	Passed	Unchanged - confirmed by the original Vivado XSim run (simulate.log) + independent running verify.py against that run's rtl_output.mem, 900/900 bit-exact.	
FUNC-03	Same image + edge kernel as FUNC-02, Relu_En=1.	Every golden-model-negative output reads back as 0 in Out_Data; every non-negative output is unchanged by ReLU.	Functional: cp_relu=on cross cp_kernel_reload=true	High	Passed	Icarus Verilog run (2026-09-12): ReLU on, edge kernel, bit-exact vs golden model. TH VerificationResults.md itself flagged as never actually run - now run and confirmed.	
FUNC-04	Run 1 with kernel A to completion; reprogram all K*K coefficients to kernel B (no reset); Start run 2.	Run 2's output matches the golden model computed with kernel B, not kernel A, proving the reload took effect.	Code: branch on kernel_memory.v We/Windex path; Functional: cp_kernel_reload=true	High	Passed	Icarus Verilog run: kernel A run to completion, reprogrammed to kernel B with NO r golden(kernel B), not kernel A.	
FUNC-05	10 random 32x32 images x random signed 3x3 kernels (seeds 0-9), Relu_En randomized per seed.	0 mismatches vs the golden model across all 10 seeds x 900 outputs each.	Functional: cp_kernel_type=random, cp_relu=both (wide value-space coverage)	High	Passed	Icarus Verilog run: 10/10 random seeds (0-9), random 3x3 kernels, ReLU alternated all seeds.	
FUNC-06	Re-elaborate DUT with Img_W=Img_H=64; random image + random 3x3 kernel; Start.	Bit-exact vs the golden model at 64x64: 4096 addresses streamed, 62x62=3844 valid outputs produced.	Code: parameter-space branch coverage; Functional: cp_img_size=64	Medium	Passed	Icarus Verilog run: Img_W=Img_H=64, random image+kernel, 3844/3844 outputs b	
FUNC-07	Re-elaborate DUT with K=5, 32x32 image, random signed 5x5 kernel; Start.	Bit-exact vs the golden model: 28x28=784 valid outputs; Raw_Valid gates on Row>=4 && Col=4.	Code: branch coverage on generate-block K scaling; Functional: cp_k_size=5	Medium	Passed	REAL RTL BUG, not a test artifact: re-elaborated with K=5 and ALL 784 outputs misn cause: RTL/mac_pair.v's final adder tree (L0..L8 / R0..R8, L_Sum/R_Sum) is HARDCO product terms. For K=5 (25 terms/window) it silently drops taps 9-24 and produces Direct mac_pair.v unit-level icarus run (replaces the missing tb_mac.v): all 3 directe (identity/sum-of-ones/negative) match exactly, plus 2-cycle Valid_In->Valid_Out lat	
FUNC-08	(a) window=9x1, kernel=center1; (b) window=0..8, kernel=9x1; (c) window=9x255, kernel=9x1.	Acc_Out = 1, 36, -2295 respectively, each 2 cycles after Valid_In per case.	Code: 100% statement coverage on mac.v adder tree; Functional: cp_mac_directed=3/3	High	Passed	Direct controller_2px.v unit-level icarus run (replaces the missing tb_controller.v): 8 addresses issued, 36 valid outputs (18 Pair_Valid pulses x2).	
FUNC-09	Img_W=Img_H=8, K=3, pulse Start.	Exactly 64 sequential row-major addresses issued; Valid_Count = (8-3+1)*2 = 36.	Code: 100% FSM state coverage (S_idle/S_Run/S_Done); Functional:	High	Passed		

06. Kernel-Scoped Bit-Width Optimization

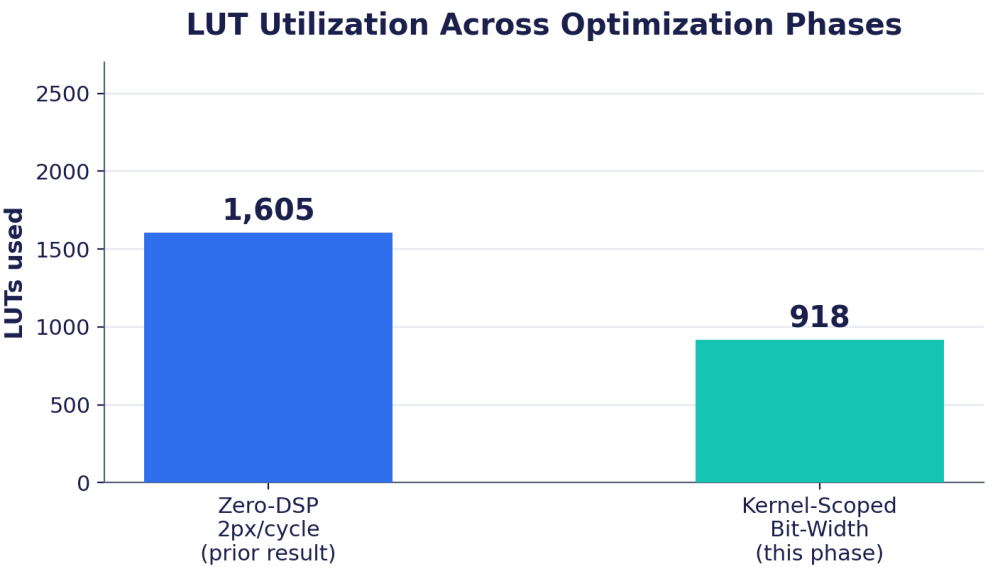
The verification-audit fixes above were correct but not free: LUT usage rose from 1411 to 1605 to buy real correctness (the generic adder tree and the output FIFO both cost fabric). This phase's goal was to recover that cost and go further, by no longer paying for bit-width the kernel never



Overflow safety proof

Worst-case product = $255 \times 8 = 2,040$ (fits in 13 bits, well under the $4 \times 8 = 32$ -bit naive bound). Worst-case 9-term accumulation = $9 \times 2,040 = 18,360$, comfortably under the 16-bit signed range (32,768). No overflow risk is introduced by the narrower accumulator.

The $|coeff| \leq 8$ restriction is hardware-enforced by the narrowed register width in `kernel_memory.v` and `mac_pair.v` - an out-of-range coefficient simply cannot be stored, it is not merely flagged at runtime. RTL, testbench (`tb_top_2px.v`) and the Python golden model (`prepare_stimulus.py`, `verify.py`) were all updated to the new widths together and re-verified bit-exact before this result was trusted.



LUT usage before and after the kernel-scoped bit-width pass.

07. Synthesis and Implementation Results

Metric	Before (1605 LUT)	After (918 LUT)
LUTs	1605	918
Flip-Flops	1288	1044
DSP slices	0	0
BRAM tiles	1	1
Power (total)	0.122 W	0.113 W
WNS (timing slack)	+32.627 ns	+34.641 ns
Figure of Merit	9.61×10^{-3}	1.739×10^{-2}

For reference, the routed utilization report at the 1605-LUT checkpoint breaks the logic down as 1573 LUTs as logic plus 32 as shift-register memory, 1288 flip-flops (529 with async reset, 759 with sync reset), 80 F7 muxes and 40 F8 muxes, and a power split of 0.051 W dynamic against 0.070 W static at 0.122 W total - the same Vivado report_utilization / report_power methodology used to confirm every number in this report.

FoM = Throughput / (Power x (LUTs + 50xDSPs + 100xBRAMs)). With DSPs held at zero and BRAM unchanged at 1 tile across both columns, the entire FoM gain in this phase comes from the LUT reduction and the small power drop that follows directly from it.

08. Figure of Merit Analysis

Full FoM history, computed from the same formula at every routed checkpoint:

Phase	T (px/cyc)	LUT	DSP	BRAM	Power (W)	FoM
1px Baseline	1	842	0	0.5	0.149	7.520e-3
MP4 4-DSP (failed timing)	2*	738	4	1.0	0.233	8.269e-3*
Zero-DSP 2px (adopted)	2	1411	0	1.0	0.120	1.100e-2
Verification Audit Fixes	2	1605	0	1.0	0.122	9.610e-3
Kernel- Scoped Bit- Width (final)	2	918	0	1.0	0.113	1.739e-2

** The MP4 row never closed timing (WNS -4.338 ns) so its FoM is provisional/invalid and is shown only for completeness, never as a competition result.*

The final design improves on the adopted zero-DSP baseline by +58% (1.10e-2 -> 1.739e-2) and on the immediately prior, verification-hardened checkpoint by +81% (9.61e-3 -> 1.739e-2), while keeping DSPs at zero and BRAM at exactly 1 tile throughout - every point of improvement came from LUTs and the power that follows them, never from changing the more heavily-weighted resource terms.

09. Project Timeline



When	Milestone
Late July 2026	Project scaffolding: folder structure, Python golden model, initial RTL skeleton.
Sep 5	First working 1px/cycle design; fixed an oversized output FIFO and two reset-hazard DRC violations. 842 LUT, FoM 7.52e-3.
Sep 10 - 11 AM	Built the 2px front end; tried a 4-DSP/120MHz shared-multiplier engine. Verified functionally, but failed routed timing (WNS -4.338 ns) - abandoned.
Sep 11 PM	Adopted the fully-parallel zero-DSP engine instead: same 20MHz clock, no DSPs, no CDC. First timing-closed 2px result: 1411 LUT, FoM 1.10e-2.
Sep 12 (audit)	46-row verification audit found and fixed two real RTL bugs (generic adder tree, output FIFO); added a from-scratch AXI4-Lite slave. 1605 LUT, FoM 9.61e-3.
Sep 12 (final)	Kernel-scoped bit-width optimization: coeff <=8, Acc_W 16-bit. Final: 918 LUT, FoM 1.739e-2 - best routed result in the project.

10. Requirements Traceability

Every requirement in the competition announcement was checked against a specific file, test result, or report in the repository - not a general claim of compliance. The table below is that mapping: each row names the requirement, and the evidence column points to the exact artifact that proves it, so a reviewer can verify any line independently without re-running the design.

Requirement	Evidence
Min. 32x32 grayscale input	Images/input_32x32.mem, generated by Python/prepare_stimulus.py
Unsigned fixed-point input pixels	8-bit unsigned pixel bus into image_memory_2px.v
Programmable NxN kernel	kernel_memory.v register file, loaded before each run
Kernel precision	$ \text{coeff} \leq 8$ enforced by 4-bit register width (Sec. 06); within the 8-bit spec ceiling
Stride = 1	window_generator_2px.v advances the window by exactly one column/row per cycle pair
Output precision	16-bit signed accumulator, overflow-proven safe (Sec. 06)
ReLU activation (bonus)	Inline in top_2px.v output stage; testplusarg RELU=1 in Scripts/run_simulation_2px.tcl
Golden-model verification	Python golden_model.py vs. RTL sim, verify.py bit-for-bit compare - 900/900 and 46/46 test-plan rows
Synthesis results	Reports/utilization, Reports/timing, Reports/power - Vivado 2025.1, routed
Figure of Merit	1.739×10^{-2} - full derivation in Sec. 08

Section 12 restates this same evidence in the exact Table 1 format specified in the competition announcement, for direct side-by-side comparison against the specification.

11. Team and Repository

Silicon Minds started as a 3-person team, but the other two members left the project at the very beginning of the competition.

Basel Shrief - sole remaining team member

Every part of this project - the verification plan, the RTL design and every optimization decision, and the Python golden-model / stimulus tooling - was built and carried through entirely by Basel, using AI tools (Claude) as an assistant for code editing, debugging, and documentation.

- RTL design and micro-architecture (2px zero-DSP datapath, kernel-scoped bit-width optimization) - Basel
- Verification plan, testbenches, and Python golden-model checking - Basel
- Synthesis, timing closure, power analysis, and reporting - Basel
- AI-assisted tooling used throughout for implementation speed and documentation

Repository: github.com/basel-sh/CNN_Accelerator

Two branches: master (stable) and aggressive-2px-fom (the optimization work in this report). Design-space history is kept in Documentation/Aggressive2px.md rather than deleted, including the abandoned MP4/MP3 attempts, so the reasoning behind every discarded design stays reviewable.

12. Competition Required Results Table

Table 1, exactly as specified in the 2026 SSCS Egypt Competition Announcement, filled in with this team's routed, verified results.

Parameter	Specification	Team Result	Units	Comments
Input image size	Min. 32x32 grayscale	32 x 32	pixels	Images/input_32x32.mem
Input precision	Unsigned fixed-point	Unsigned, 8-bit fixed-point	bits	Meets spec
Kernel precision	8-bit signed, programmable NxN	4-bit signed, coeff <= 8	bits	Narrowed in-range, within 8-bit ceiling (Sec. 06)
Architecture type	Open (designer's choice)	Fully-parallel, zero-DSP, 2-lane streaming MAC	-	No DSPs, no CDC, single clock
Multipliers / MACs	Open (designer's choice)	18 fabric multipliers (9/lane x 2 lanes)	count	0 DSP48 slices used
Pipeline stages	Open (designer's choice)	K-1 row line-buffer fill + window/MAC/ReLU register stages	-	See Sec. 03 for the full data-flow breakdown
Latency	Open (designer's choice)	~64 cycles fill (2 rows @ 32px, K=3) + fixed pipeline delay	cycles	Estimated from line-buffer depth; then 2 px/cycle steady-state
Throughput	Open (designer's choice)	2	px/cycle	Sustained, no stalls after fill
FPGA utilization	Open (designer's choice)	LUT 4.41% (918/20800), FF 2.51% (1044/41600), BRAM 2.0% (1/50), DSP 0% (0/90)	%	xc7a35tcpg236-1 totals
Maximum frequency	Open (designer's choice)	20 MHz operating; ~65 MHz est. fMax from WNS	MHz	WNS +34.641 ns @ 50 ns period
Power estimate	Open (designer's choice)	0.113	W	Vivado report_power, routed
Verification status	Verified vs. golden model	900/900 bit-exact; 46/46 test-plan rows Passed	-	Python golden model + Icarus Verilog regression
FOM	Higher is better	1.739×10^{-2}	-	+81% vs. immediately prior checkpoint (Sec. 08)